Comandos SQL:

Criar tabela SQL

```
CREATE TABLE Teste (
       Nome VARCHAR(100) NOT NULL CHECK (NOT (nome REGEXP '[^a-z ''àáãâéêíóõôúçñ-]')) COMMENT 'Nome co
       Estado enum('confirmado', 'pendente') not null,
       Data_Hora datetime not null comment 'Data e hora em que ocorreu a intervenção/reparação',
       NIF CHAR(9) not null CHECK (NIF REGEXP '^[0-9]{9}$'),
       Min DECIMAL(5, 2) NOT NULL,
       Latitude DECIMAL(8, 6) NOT NULL,
       CustoIntervencao Decimal(10,2) not null CHECK(CustoIntervencao >= Min),
       Matricula varchar(8) not null CHECK (Matricula REGEXP '^[A-Z]{2}-[0-9]{2}-[0-9]{2}$'
                  OR Matricula REGEXP '^[0-9]{2}-[0-9]{2}-[A-Z]{2}$'
                  OR Matricula REGEXP '^[0-9]{2}-[A-Z]{2}-[0-9]{2}$') COMMENT 'Matrícula do veículo',
       PRIMARY KEY (Matricula),
       foreign key (Matricula) references Veiculo(Matricula)
           ON DELETE CASCADE
           ON UPDATE CASCADE
   )COMMENT='Teste';
```

SELECT:

Listar os nomes das pessoas com **mais de 24 opiniões** nas **últimas 24 horas** ordenadas por idade.

```
SELECT
    p.nomeCompleto,
    p.dataNascimento,
    COUNT(o.PId) AS totalOpinioes
FROM
    Pessoa p
JOIN
    Opiniao o ON p.PId = o.PId
WHERE
    o.dataHoraSubmissao >= NOW() - INTERVAL 1 DAY -- Últimas 24 horas
GROUP BY
    p.PId
HAVING
    COUNT(o.PId) > 24 -- Mais de 24 opiniões
ORDER BY
    p.dataNascimento ASC;
```

Listar a **quantidade** de clientes que ocupa cada uma das mesas identificando primeiro as mesas livres

*Aqui o **LEFT JOIN** dá prioridade ao M.idMesa fazendo com que preencha a **0** caso a mesa esteja livre*

```
SELECT
    M.idMesa, COUNT(O.NIFCliente) AS QuantidadeClientes
FROM
    Mesa M
LEFT JOIN
    Ocupar O ON M.idMesa = O.idMesa
GROUP BY
    m.idMesa
ORDER BY
    QuantidadeClientes ASC;
```

Listar as temperaturas **máximas** por **mês**.

```
SELECT
    EXTRACT(YEAR FROM Dia) AS Ano,
    EXTRACT(MONTH FROM Dia) AS Mes,
    MAX(Max) AS Temperatura_Maxima
FROM
    Temperatura
GROUP BY
    EXTRACT(YEAR FROM Dia),
    EXTRACT(MONTH FROM Dia)
ORDER BY
    Ano, Mes;
```

Listar as marcas com **mais avarias** pendentes.

```sql
SELECT Marca, COUNT(*) AS TotalAvariasPendentes
FROM Avaria
WHERE Estado = 'pendente'
GROUP BY Marca
ORDER BY TotalAvariasPendentes DESC;
```

Listar para cada contribuinte com **mais de 38 anos**, a quantidade de propriedades registadas atualmente.

```sql
SELECT
    c.nome AS nome_contribuinte,
    COUNT(r.propId) AS quantidade_propriedades
FROM
    Contribuinte c
LEFT JOIN
    Registo r ON c.NIF = r.NIF
WHERE
    TIMESTAMPDIFF(YEAR, c.dataNascimento, CURDATE()) > 38
GROUP BY
    c.NIF
ORDER BY
    quantidade_propriedades DESC, nome_contribuinte;
```

Gerar a lista dos eletrodomésticos com **2 avarias** ocorridas no **mês em que foram comprados**.

```sql
SELECT a.NumeroSerie, e.Marca, COUNT(*) AS TotalAvarias
FROM Avaria a
JOIN Eletrodomestico e ON a.NumeroSerie = e.NumeroSerie
WHERE MONTH(a.DataAvaria) = MONTH(e.DataVenda)
  AND YEAR(a.DataAvaria) = YEAR(e.DataVenda)
GROUP BY a.NumeroSerie, e.Marca
HAVING COUNT(*) = 2;
```

INSERT:

Registar um aluguer do 'Joe Six-Pack' desde o **inicio ao fim da próxima semana**.

```sql
INSERT INTO Aluguer (Cliente_NIF, DtHoraInicio, DtFim)
VALUES (
    (SELECT NIF FROM Cliente WHERE nome = 'Joe Six-Pack'),
    DATE_ADD(CURRENT_DATE, INTERVAL (8 - DAYOFWEEK(CURRENT_DATE)) DAY), -- Início da próxima semana
    DATE_ADD(CURRENT_DATE, INTERVAL (14 - DAYOFWEEK(CURRENT_DATE)) DAY) -- Fim da próxima semana (pr
);
```

Registar a temperatura de **ontem** no ISEL.

```sql
INSERT INTO Temperatura (Latitude, Longitude, Dia, Min, Max)
VALUES (
    (SELECT Latitude FROM Local WHERE Designacao = 'ISEL'),
    (SELECT Longitude FROM Local WHERE Designacao = 'ISEL'),
    CURDATE() - INTERVAL 1 DAY, -- Data de ontem
    10.5, -- Temperatura mínima
    15.2  -- Temperatura máxima
);
```

Adicionar **ontem à meia-noite** uma segunda avaria nos eletrodomésticos com números de serie **iniciados por 'fev-2022'**.

*-O **CURDATE()** retorna apenas a data atual, o **NOW()** retorna data e hora atual.*

*-O **"="** apenas permite procurar por uma palavra exata, o **"LIKE"** permite usar funções como o % para procurar qualquer valor depois disso.*

```
INSERT INTO Avaria (Marca, NumeroSerie, DataAvaria, Descricao, Estado)
SELECT Marca, NumeroSerie, DATE_FORMAT(DATE_SUB(NOW(), INTERVAL 1 DAY), '%Y-%m-%d 00:00:00') AS DataAvaria,
       'Segunda avaria registrada automaticamente' AS Descricao,
       'pendente' AS Estado
FROM Avaria
WHERE NumeroSerie LIKE 'fev-2022%';
```

DELETE:

Eliminar as avarias ocorridas **à mais de 2 anos**. ||| **Eliminar** assuntos criados no **milénio passado**

```
DELETE FROM Avaria
WHERE DataAvaria > DATE_ADD(Eletrodomestico.DataVenda, INTERVAL 2 YEAR)
AND Avaria.NumeroSerie = Eletrodomestico.NumeroSerie;
```

```
DELETE FROM Assunto
WHERE YEAR(dataHoraCriacao) < 2000;
```

Apagar o **registo** de todas as **propriedades** localizadas em 'Mêda'.

```
DELETE FROM Registo
WHERE propId IN (
    SELECT propId
    FROM Propriedade
    WHERE cidade = 'Mêda'
);
```

UPDATE:

Assinalar todas as avarias ocorridas no **último fim de semana** como resolvidas.

(WEEKDAY(CURDATE()) + 2) → *Indica o Sábado* ***(WEEKDAY(CURDATE()) + 6)*** → *Indica o Domingo*

```
UPDATE Avaria
SET Estado = 'resolvido'
WHERE DataAvaria BETWEEN CURDATE() - INTERVAL (WEEKDAY(CURDATE()) + 2) DAY
                     AND CURDATE() - INTERVAL WEEKDAY(CURDATE()) DAY
  AND Estado != 'resolvido';
```

VIEWS:

Criar uma vista que liste os técnicos com **menos avarias** resolvidas.

```
Create view ListaTecnicos As
Select
    T.Nome, T.NIF, Count(A.DataAvaria) AS AvariasResolvidas
from tecnico T
join TecnicoIntervencao TI ON TI.NIF=T.NIF
join intervencao I ON I.DataHoraIntervencao = TI.DataHoraIntervencao
join Avaria A ON A.DataAvaria = I.DataAvaria
where A.Estado = 'resolvido'
Group By T.NIF, T.Nome
order by AvariasResolvidas ASC;
```

Criar vista para listar a informação sobre os veículos alugados no **trimestre passado**.

```sql
CREATE VIEW InfoVeiculosTrimestrePassado AS
SELECT
    V.matricula,V.marca,V.modelo,V.DtRegisto,A.DtHoraInicio,A.DtFim,
FROM
    Veiculo V,
JOIN
    AluguerVeiculo AV ON AV.Matricula=A.Matricula,
JOIN
    Aluguer A ON A.DtHoraInicio=AV.DtFim,
WHERE
    A.DtFim >= DATE_TRUNC('quarter', CURDATE()) - INTERVAL '6 months'
    AND A.DtFim < DATE_TRUNC('quarter', CURDATE()) - INTERVAL '3 months';
```

Criar vista para calcular o **tempo médio** de ocupação das mesas

```sql
CREATE VIEW TempoOcupacao AS
SELECT
    idMesa, AVG(TIMESTAMPDIFF(MINUTE, DataHoraEntrada, DataHoraSaida)) AS TempoMedioOcupacao
FROM
    Cliente_Mesa
GROUP BY
    idMesa;
```

Tipos de Operações SQL:

1. COUNT()

- **Descrição**: Conta o número de linhas ou o número de valores não nulos em uma coluna.
- **Exemplo**:

sql — Copiar código

```sql
SELECT COUNT(*) FROM tabela;  -- Conta todas as linhas
SELECT COUNT(coluna) FROM tabela;  -- Conta as linhas onde a coluna não é nula
```

3. AVG()

- **Descrição**: Calcula a média dos valores de uma coluna numérica.
- **Exemplo**:

```sql
SELECT AVG(coluna) FROM tabela;
```

4. MIN()

- **Descrição**: Retorna o valor mínimo de uma coluna.
- **Exemplo**:

```sql
SELECT MIN(coluna) FROM tabela;
```

5. MAX()

- **Descrição**: Retorna o valor máximo de uma coluna.
- **Exemplo**:

```sql
SELECT MAX(coluna) FROM tabela;
```

2. SUM()

- **Descrição**: Soma todos os valores de uma coluna numérica.
- **Exemplo**:

```sql
SELECT SUM(coluna) FROM tabela;
```

Tipos de JOIN:

1. INNER JOIN

- **Descrição:** Retorna apenas as linhas que têm correspondências nas duas tabelas.
- **Exemplo:** Combina clientes e seus pedidos, mas exclui clientes que não têm pedidos.

sql — Copiar código

```sql
SELECT *
FROM TabelaA
INNER JOIN TabelaB
ON TabelaA.chave = TabelaB.chave;
```

4. FULL JOIN (ou FULL OUTER JOIN)

- **Descrição:** Retorna todas as linhas de ambas as tabelas, combinando as correspondências e preenchendo com `NULL` onde não há correspondência.
- **Exemplo:** Lista todos os clientes e todos os pedidos, mesmo que não estejam associados.

sql — Copiar código

```sql
SELECT *
FROM TabelaA
FULL JOIN TabelaB
ON TabelaA.chave = TabelaB.chave;
```

2. LEFT JOIN (ou LEFT OUTER JOIN)

- **Descrição:** Retorna todas as linhas da tabela à **esquerda** (TabelaA), mesmo que não haja correspondência na tabela à direita (TabelaB). Para linhas sem correspondência, os valores da tabela à direita aparecem como `NULL`.
- **Exemplo:** Lista todos os clientes, incluindo aqueles sem pedidos.

sql — Copiar código

```sql
SELECT *
FROM TabelaA
LEFT JOIN TabelaB
ON TabelaA.chave = TabelaB.chave;
```

3. RIGHT JOIN (ou RIGHT OUTER JOIN)

- **Descrição:** Retorna todas as linhas da tabela à **direita** (TabelaB), mesmo que não haja correspondência na tabela à esquerda (TabelaA). Para linhas sem correspondência, os valores da tabela à esquerda aparecem como `NULL`.
- **Exemplo:** Lista todos os pedidos, incluindo aqueles que não têm cliente associado.

sql — Copiar código

```sql
SELECT *
FROM TabelaA
RIGHT JOIN TabelaB
ON TabelaA.chave = TabelaB.chave;
```

5. CROSS JOIN

- **Descrição:** Retorna o **produto cartesiano** das tabelas, combinando todas as linhas da tabela à esquerda com todas as linhas da tabela à direita.
- **Exemplo:** Se uma tabela tem 3 linhas e a outra tem 4, o resultado terá 12 linhas.

sql — Copiar código

```sql
SELECT *
FROM TabelaA
CROSS JOIN TabelaB;
```

6. NATURAL JOIN

- **Descrição:** É um tipo de **INNER JOIN** que combina automaticamente as tabelas com base em colunas com o mesmo nome e tipo.
- **Exemplo:** Se as tabelas têm uma coluna chamada `id` em comum, ele realiza o join baseado nela.

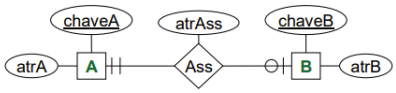
sql — Copiar código

```sql
SELECT *
FROM TabelaA
NATURAL JOIN TabelaB;
```

Conversão Modelo EA para Relacional:

Associação binária 1:1 (1-1 : 0-1)

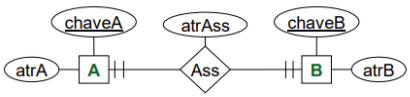
- Participação obrigatória apenas num dos lados
  - Neste caso optou-se por participação obrigatória ** do lado da entidade B



- Cada entidade é modelada por um Esquema de Relação (EsqRel), em que o EsqRel da entidade do lado obrigatório referencia o outro EsqRel
  - A(chaveA, atrA)
    - Chaves Candidatas = { chaveA }
  - B(chaveB, atrB, atrAss, chaveA)
    - Chaves Candidatas = { chaveB } e { chaveA }
    - Chaves Estrangeiras* = { chaveA }
- Esta configuração modela melhor o cenário do que a configuração anterior, pois a participação obrigatória é imposta

Associação binária 1:1 (1-1 : 1-1)

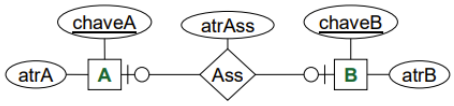
- Obrigatória em ambos os lados



- As duas entidades são juntas num só EsqRel
  - AB(chaveA, atrA, atrAss, chaveB, atrB)
    - Chaves Candidatas = { chaveA } e { chaveB }
- Alternativo: cada entidade fica num EsqRel, e um qualquer deles (EsqRel[1]) referencia o outro (EsqRel[2])
  - Chaves Candidatas de EsqRel[1] = { chaveA } e { chaveB }

Associação binária 1:1 (1-0 : 0-1)

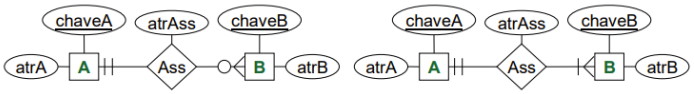
- Nenhum dos lados é obrigatório



- A associação é modelada por um EsqRel próprio
  - A(chaveA, atrA)
    - Chaves Candidatas = { chaveA }
  - B(chaveB, atrB)
    - Chaves Candidatas = { chaveB }
  - A_B (chaveA, atrAss, chaveB }
    - Chaves Candidatas = { chaveA } e { chaveB }
    - Chaves Estrangeiras = { chaveA } e { chaveB }

Associação binária 1:N (1-1 : ?-N)

- Participação obrigatória do lado N* (indiferente do lado 1)

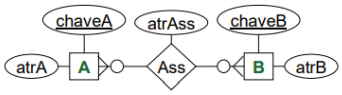


- O EsqRel do lado N, referencia o outro EsqRel
  - A(chaveA, atrA)
    - Chaves Candidatas = { chaveA }
  - B(chaveB, atrB, atrAss, chaveA)
    - Chaves Candidatas = { chaveB }
    - Chaves Estrangeiras = { chaveA }
- A situação do lado 1 terá que ser garantida por processamento adicional

* Obrigatória do lado N, entenda-se que cada entidade do lado N tem participação obrigatória na associação

Associação binária M:N (M-0 : 0-N)

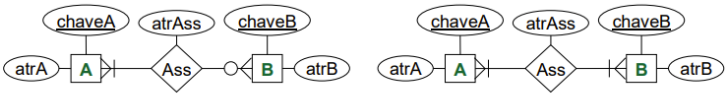
- Facultativo nos dois lados



- A associação é modelada com um EsqRel próprio
  - A(chaveA, atrA)
    - Chaves Candidatas = { chaveA }
  - B(chaveB, atrB)
    - Chaves Candidatas = { chaveB }
  - A_B (chaveA, chaveB, atrAss)
    - Chaves Candidatas = { chaveA, chaveB }
    - Chaves Estrangeiras = { chaveA } e { chaveB }

Associação binária M:N (M-$?_1$:$?_2$-N) ($?_1$ or $?_2$ = 1)

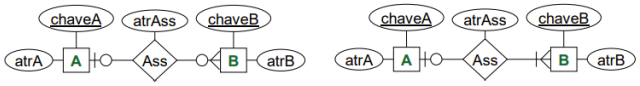
- Obrigatoriedade num ou nos dois lados



- O Esquema Relacional é o mesmo que o do caso em que não existe obrigatoriedade em nenhum dos lados
- No entanto é preciso notar que a garantia da participação obrigatória não é conseguida directamente pelo Esquema Relacional, necessitando de processamento adicional

Associação binária 1:N (1-0 : ?-N)

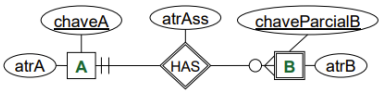
- Nenhum dos lados é obrigatório e obrigatório do lado 1



- A associação é modelado com um EsqRel próprio
  - A(chaveA, atrA)
    - Chaves Candidatas = { chaveA }
  - B(chaveB, atrB)
    - Chaves Candidatas = { chaveB }
  - A_B (chaveB, atrAss, chaveA)
    - Chaves Candidatas = { chaveB }
    - Chaves Estrangeiras = { chaveB } e { chaveA }
- Na situação de obrigatório do lado 1 :
  - tal tem de ser assegurado por processamento.
- A associação também poderia ser modelada de forma idêntica ao slide anterior, caso a chave estrangeira possa ser NULL. Situação análoga à do slide 11.

Entidade Fraca

- Independentemente da obrigatoriedade do lado da entidade fraca



- A entidade fraca é modelada com um EsqRel que conterá também a chave do EsqRel da entidade da qual depende
  - A(chaveA, atrA)
    - Chaves Candidatas = { chaveA }
  - B(chaveParcialB, chaveA, atrB, atrAss)
    - Chaves Candidatas = { chaveParcialB, chaveA }
    - Chaves Estrangeiras = { chaveA }

Normalização:

1ª Forma Normal (1NF)

Critério:

- Cada coluna deve conter valores atômicos (não divisíveis).
- Não podem existir grupos repetidos ou arrays dentro de uma coluna.
- Cada registro (linha) deve ser único.

Exemplo (antes da 1NF):

ID Cliente	Nome	Telefones
1	João Silva	912345678, 932345678

Após normalização (1NF):

ID Cliente	Nome	Telefone
1	João Silva	912345678
1	João Silva	932345678

Segunda Forma Normal



Uma tabela está na 2ª FN se:

- Está na 1ªFN
- Todos os atributos não-chave são funcionalmente dependentes de todas as partes da chave primária.
- Não existem dependências parciais.
- Caso contrário, deve-se gerar uma nova tabela com os dados.

Um atributo-chave é um que é uma PK ou parte de uma PK composta.

Exemplo (antes da 2NF):

ID Pedido	ID Cliente	Nome Cliente	Data Pedido
101	1	João Silva	2025-01-10
102	1	João Silva	2025-01-11

Problema: O nome do cliente depende apenas do `ID Cliente`, não do `ID Pedido`.

Após normalização (2NF): Tabela Clientes

ID Cliente	Nome Cliente
1	João Silva

Tabela Pedidos

ID Pedido	ID Cliente	Data Pedido
101	1	2025-01-10
102	1	2025-01-11

3ª Forma Normal (3NF)

Critério:

- A tabela deve estar na 2NF.
- Não deve haver dependências transitivas (ou seja, uma coluna depende de outra coluna que não é a chave primária).

Exemplo (antes da 3NF):

ID Cliente	Nome Cliente	Cidade	Estado
1	João Silva	Lisboa	Lisboa

Problema: `Estado` depende de `Cidade`, e não diretamente de `ID Cliente`.

Após normalização (3NF): Tabela Clientes

ID Cliente	Nome Cliente	Cidade
1	João Silva	Lisboa

Tabela Cidades

Cidade	Estado
Lisboa	Lisboa

4. **Forma Normal de Boyce-Codd (BCNF)**:

- R está em BCNF se, para cada dependência funcional $X \rightarrow Y$, X for uma superchave (isto é, X contém uma chave candidata).
- Verificamos as dependências em F:
  - $\{A3, A4\} \rightarrow \{A1, A4\}$: $\{A3, A4\}$ é uma superchave, pois é uma chave candidata.
  - $\{A4, A1\} \rightarrow A3$: $\{A4, A1\}$ também é uma superchave, pois determina todos os atributos.
- Todas as dependências respeitam a condição de superchave. Assim, R está em BCNF.

Exemplo para Entendimento

Considere a relação R com o seguinte esquema:

$$R(AlunoID, Curso, Professor)$$

E as dependências funcionais:

1. $\text{AlunoID} \rightarrow \text{Curso}$
2. $\text{Curso} \rightarrow \text{Professor}$

Passo 1: Chaves Candidatas

- A única **chave candidata** é $\{AlunoID, Curso\}$, pois determina todos os atributos da relação.

Passo 2: Avaliação da BCNF

- Para $\text{AlunoID} \rightarrow \text{Curso}$:
  - AlunoID não é uma superchave, pois não determina todos os atributos de R.
  - **R não está em BCNF.**

4ª Forma Normal (4NF):

- Uma relação está em 4NF se:
  1. Está em BCNF.
  2. Não contém dependências multivaloradas, exceto aquelas que envolvem uma superchave.
- **Dependência multivalorada** ocorre quando um atributo depende de outro, mas independentemente de outros atributos.
- Exemplo:
  - Não em 4NF: `Aluno(ID, Curso, AtividadeExtracurricular)` onde um aluno pode ter vários cursos e várias atividades independentes.
  - Em 4NF: Dividir em:
    - `AlunoCurso(ID, Curso)`
    - `AlunoAtividade(ID, AtividadeExtracurricular)`.

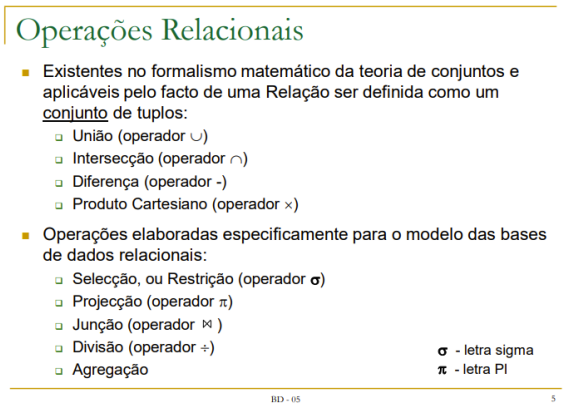
5ª Forma Normal (5NF):

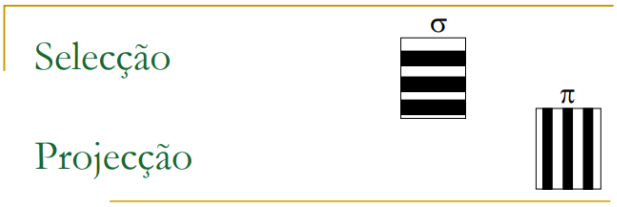
- Uma relação está em 5NF (ou Forma Normal de Projeção e Junção) se:
  1. Está em 4NF.
  2. Não contém dependências de junção que não sejam implicadas pelas chaves candidatas.
- Exemplo:
  - Relacionamentos complexos podem ser divididos em várias tabelas e depois reconstruídos sem perda de dados.

6ª Forma Normal (6NF):

- Raramente usada, aplica-se a sistemas temporais.
- Cada dependência funcional deve ser representada como uma tabela separada.

Álgebra Relacional:





Escrever uma expressão em álgebra relacional que permita obter a 'Designação' dos locais com temperaturas Min = Max-3 na **semana passada**.

$$\pi_{\text{Designação}}\left(\sigma_{\text{Min}=\text{Max}-3 \,\wedge\, \text{Dia}\in[D1,D7]}(\text{Temperatura}) \bowtie \text{Local}\right)$$

Expressão álgebra relacional, para obter as cores dos produtos fornecidos no **trimestre passado**

```
π(cor) (σ(dataPrevistaFornecimento ≥ DATA_INICIO_TRIMESTRE ∧ dataPrevistaFornecimento ≤ DATA_FIM_TRIMESTRE) (Produto ⋈ Fornece))
```

Obter o nome dos clientes que festejam **hoje** o seu aniversário.

$$\pi_{Nome}\left(\sigma_{\text{dia(dataNascimento)} = \text{dia(hoje)} \,\wedge\, \text{mes(dataNascimento)} = \text{mes(hoje)}}(\text{Cliente})\right)$$

Obter os Títulos dos livros cujos Autores nasceram no **século XVI**.

```
π Título (σ (EXTRACT(YEAR FROM dataNascimento) >= 1501 ∧ EXTRACT(YEAR FROM dataNascimento) <= 1600) (AutorPrincipal) ⋈ Livro)
```

Recuperar R a partir de D:

Expressão em álgebra relacional para **recuperar 'R'** a partir de D={**R1(A3,A4,A2)**, **R2(A4,A1,A3)**}

$$R = \pi_{A1,A2,A3,A4}\left((R1 \bowtie_{R1.A3=R2.A3 \wedge R1.A4=R2.A4} R2)\right)$$